Le langage Python

Introduction à Python

Dominique FRANCISCI

(2020-2026)



Objectifs de ce support

- Connaître les origines du langage Python
- Etre capable de comprendre le langage Python
- Etre capable d'écrire (d'implanter/d'implémenter) des programmes en Python

Sommaire



- **Origines du langage Python**
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie

Origines du langage Python

… mais qu'est ce qu'un langage de programmation ?

Un langage de programmation est construit à partir :

- D'une grammaire formelle :
  - Symboles
  - Règles syntaxiques
- De règles sémantiques

- Un langage de programmation permet d'écrire :
  - Des structures de données
  - Les traitements sur ces données

Origines du langage Python





Le créateur de Python :

- Guido van Rossum
- Ingénieur néerlandais né en 1956
- Université d'Amsterdam
- Fan de Monty Python's Flying Circus → d'où Python
- Benevolent Dictator for Life (« dictateur bienveillant à vie ») jusqu'en juillet 2018 au sein de la communauté pythonienne

MONTY PYTHON'S FLYING CIRCUS

Origines du langage Python



Le langage Python :

- Première version le 20 février 1991
- Langage de haut niveau à usage général
- Langage multiparadigme et multiplateforme
- Très expressif et comporte une syntaxe simple
- Conçu pour la relecture du code
- Langage dynamique et interprété
- L'indentation est interprétée → écriture plus lisible
- Conçu pour optimiser la productivité des développeurs
- Plusieurs versions : Python 3.x, Pyhton 2.y

Où en sommes-nous ?



- Origines du langage Python
- **Informations pratiques**
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie

Informations pratiques

Domaines d'application

Très nombreux (grâce à ses nombreuses bibliothèques) :

- Monde scientifique
- Intelligence Artificielle (réseaux de neurones, …)
- Milieu pédagogique
- Mise au point de prototypes
- Automatisation de tâches (langage script)
- …

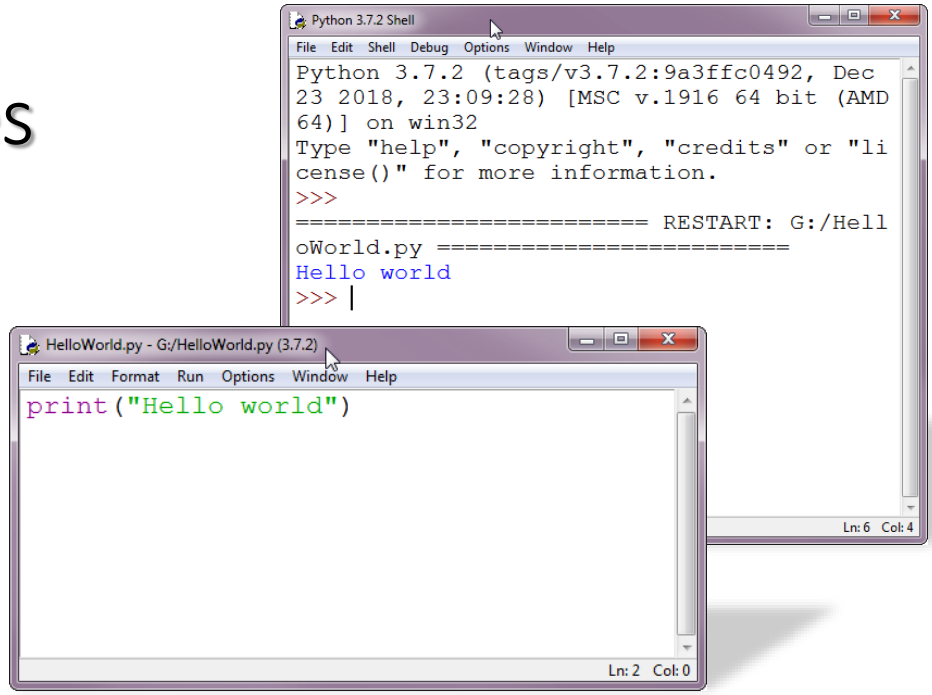
Informations pratiques

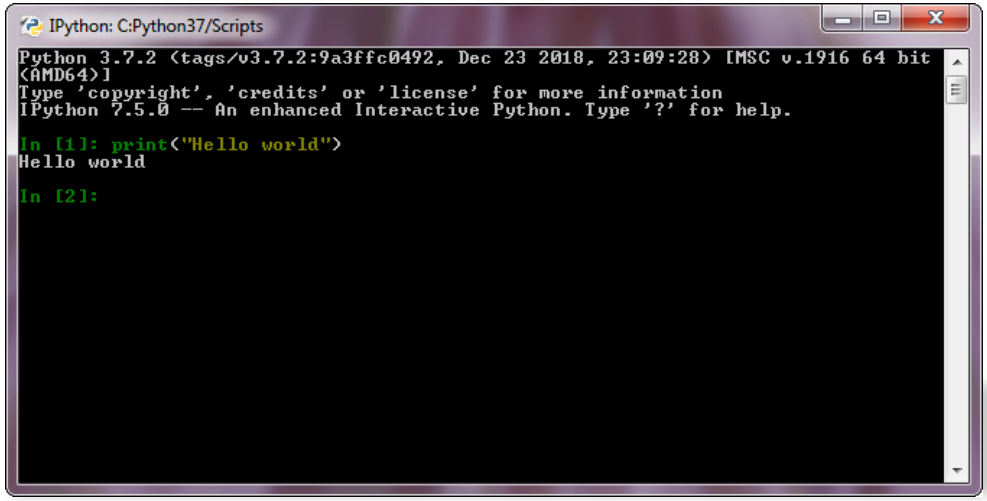
Interpréteurs :

- Python IDLE :
  - Basique
  - Distribué avec Python
  - Interpréteur de référence

- IPython :

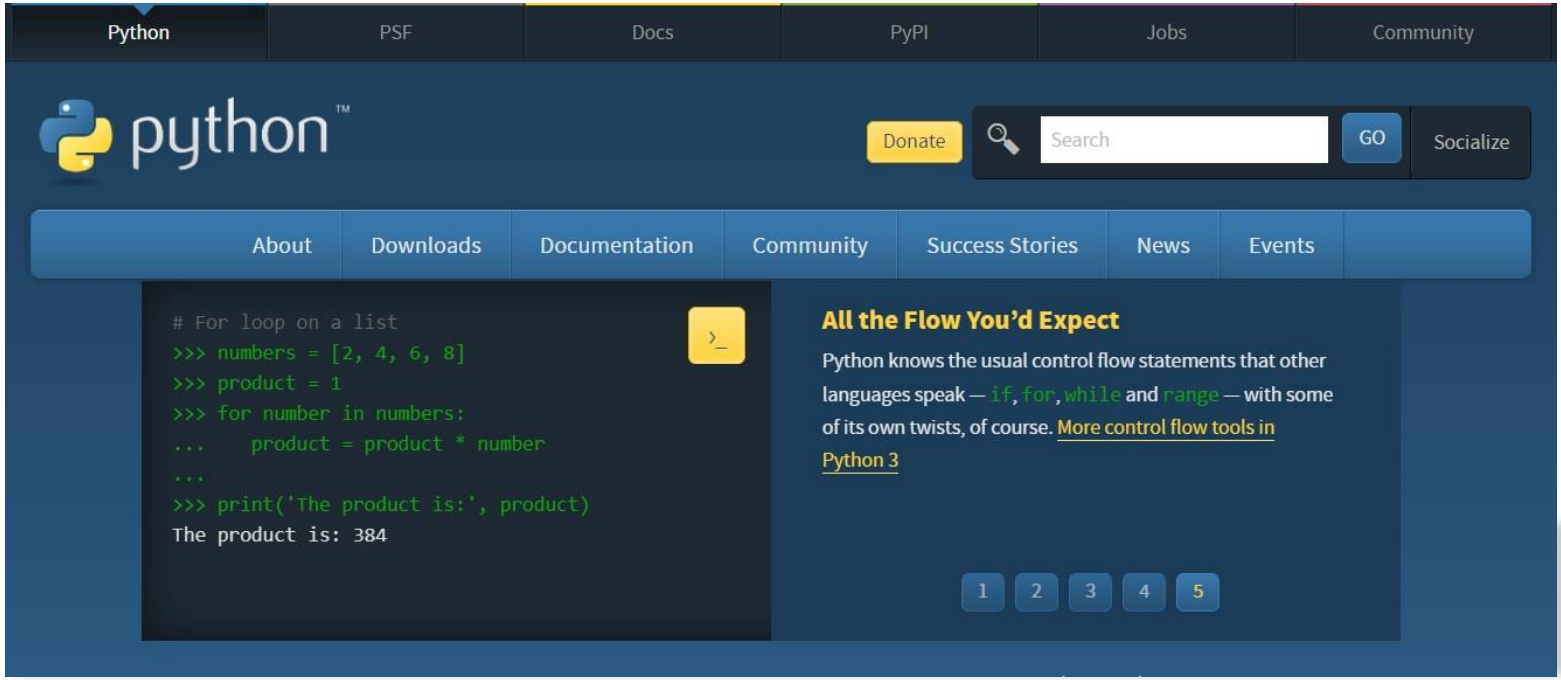
  Fonctionnalités avancées interactives :
  - Auto-completion
  - Sauvegarde de la session
  - Coloration du code
  - Edition multiligne





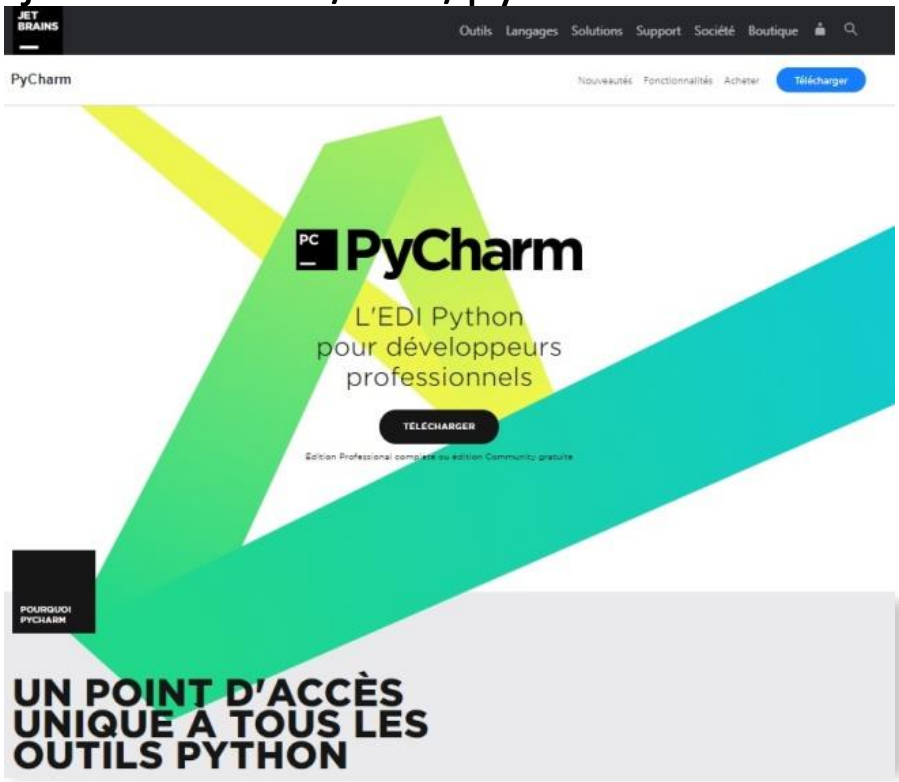
Informations pratiques

- Site web officiel Python :
  https://www.python.org
- Dernière version de Python : 3.13



Informations pratiques

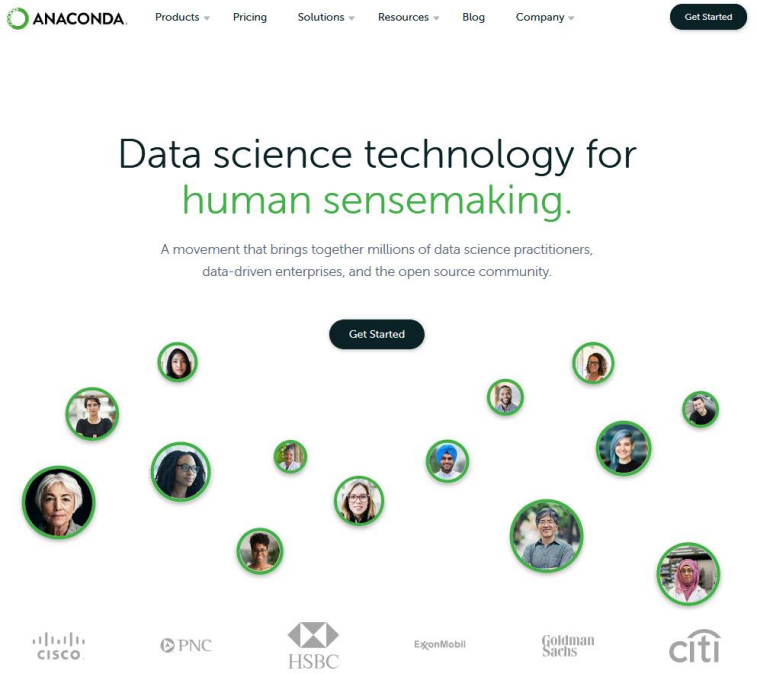
- EDI (Environnement de Développement Intégré) PyCharm : https://www.jetbrains.com/fr-fr/pycharm

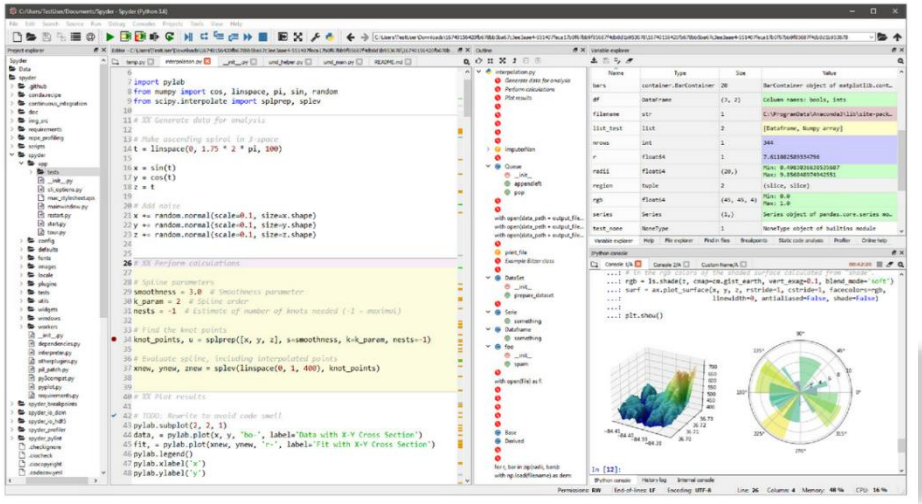


Informations pratiques

- Anaconda & EDI Spyder :

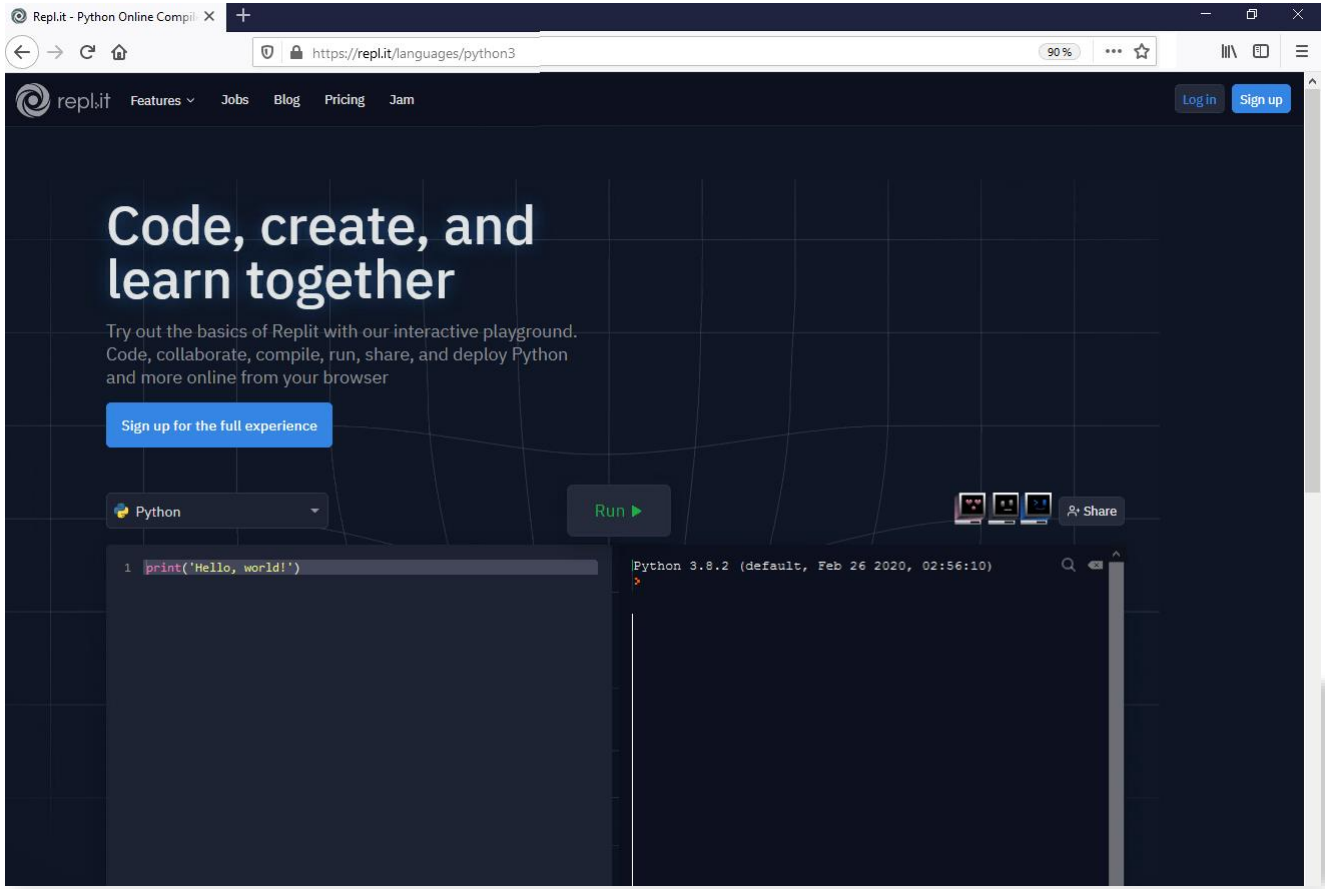
  https://https://www.anaconda.com/



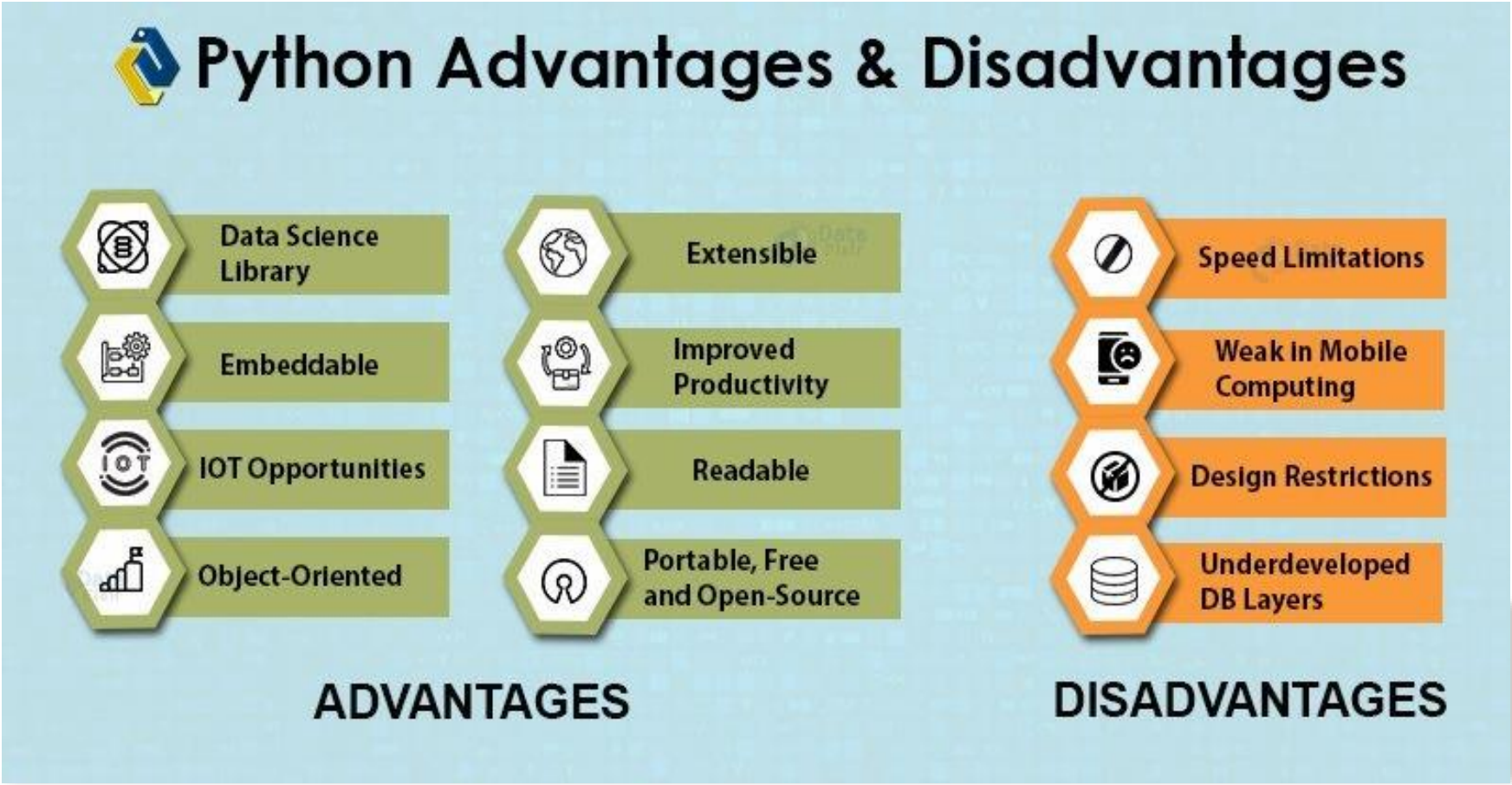


Informations pratiques

Il existe des environnements de développement en ligne



Informations pratiques



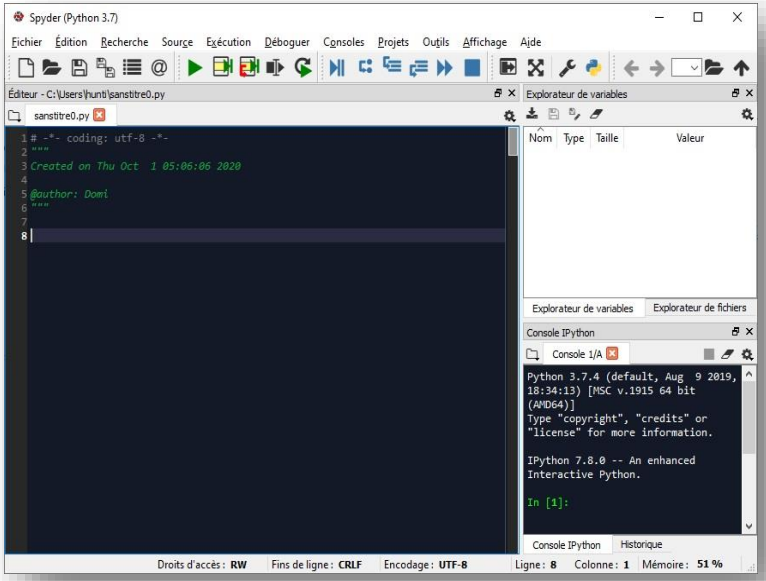
Informations pratiques

- Comment exécuter l'interpréteur Python ?
- Nous considérons par exemple, l'EDI Spyder

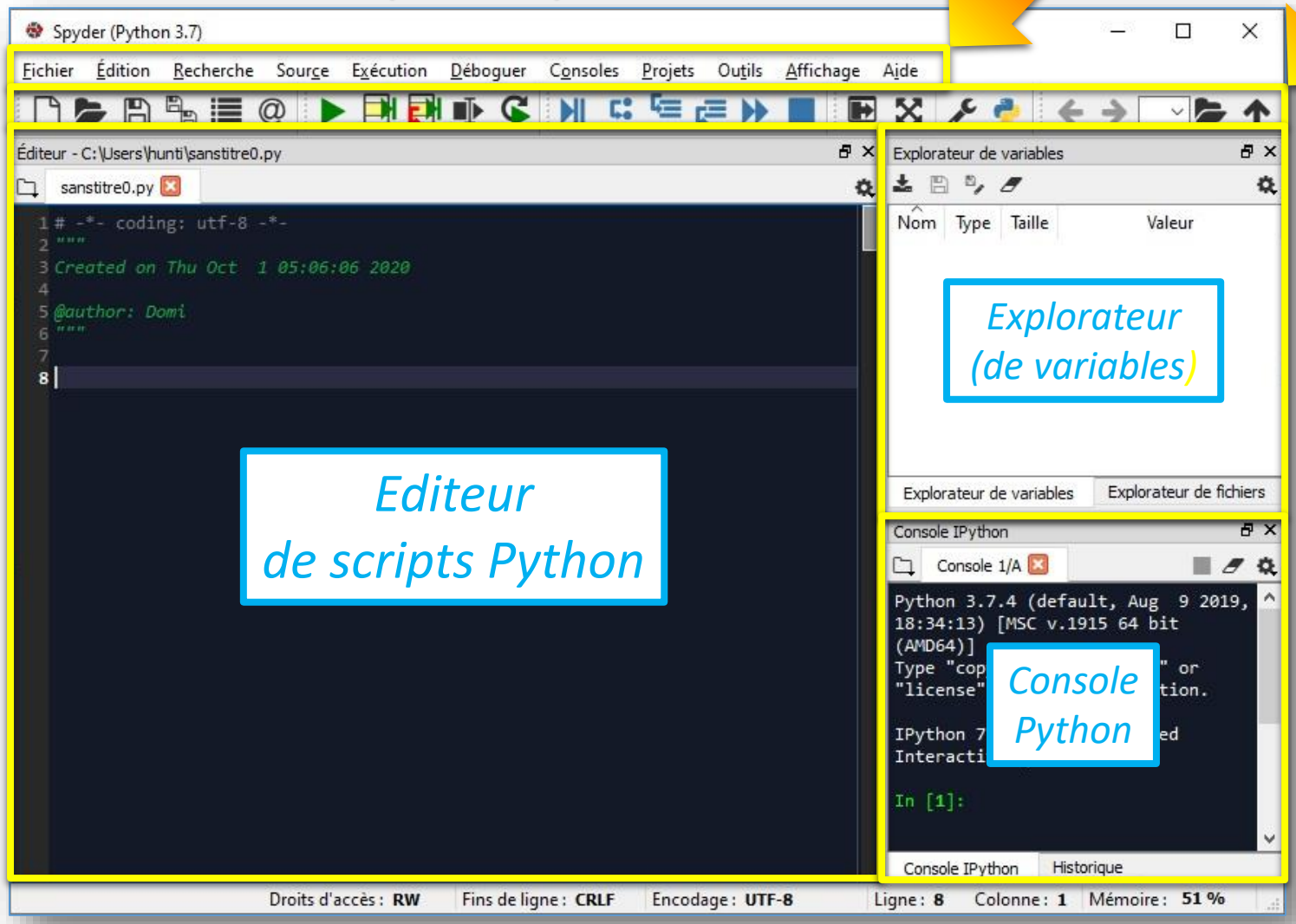


1 - Lancement

1 – EDI Spyder



Informations pratiques



Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- **Données informatiques**
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie

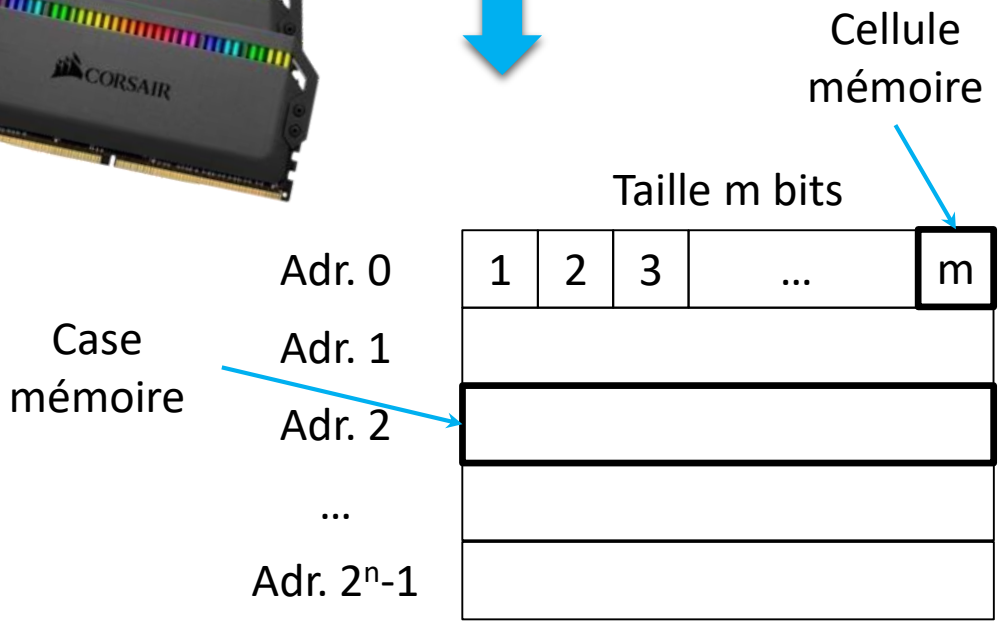


Données informatiques

- Les données sont stockées dans des cases mémoires
- Elles sont accessibles via des adresses

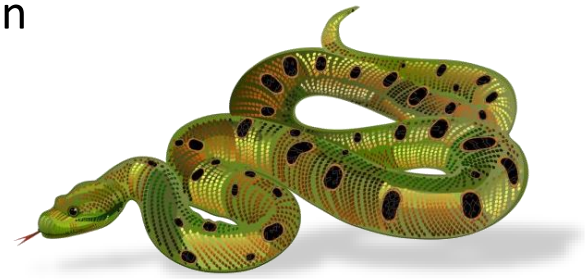






Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- **Commentaires**
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie



Commentaires

Syntaxe : #

- Tout ce qui suit ce symbole n'est pas interprété
- Equivalent au // en C, C++ et Java
- Pas d'équivalence du /* */



Exemple

```
>>> # Un premier commentaire
>>> x = 1 # Un autre commentaire
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- **Types de données standards**
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie



Types de données standards

- Python permet de manipuler plusieurs sortes de données (type) :
  - Entiers : `-10, 0, 39, 1945` Type `int`
  - Flottants (réels) : `-10.5, 3.14159265, 1974.5` Type `float`
  - Complexes : `-1.5 + 3.8j` Type `complex`
  - Booléens : `True, False` (vrai, faux) Type `bool`
  - Chaînes de caractère : `"Le langage Python"` Type `str`

- On utilise la fonction **`type`** pour connaître le type d'une donnée
- On utilise la fonction **`print`** pour afficher une donnée
- On utilise la fonction **`help`** pour imprimer l'aide sur un symbole

Exemple



```
>>> type(3)
  <class 'int'>
>>> print("Les sanglots longs des violons de l'automne")
  Les sanglots longs des violons de l'automne
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- **Entiers/flottants**
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie



Entiers/flottants

Le type « int »

- Permet de représenter n'importe quel entier positif ou négatif
- On utilise l'écriture décimale du nombre (base 10 : 0, 1, …, 9)
- Il est possible d'utiliser les représentations : binaire (base 2 : 0,1), octale (base 8 : 0, 1, …, 7) ou hexadécimale (base 16 : 0, 1, …, 9, A, B, …, F)

Exemple



```
>>> type(-1974)
  <class 'int'>
>>> 0b11110110110 # Ecriture binaire
  1974
>>> 0o3666        # Ecriture octale
  1974
>>> 0x7B6         # Ecriture hexadécimale
  1974
```

Entiers/flottants

Le type « float »

- Permet de représenter une partie des nombres à virgule
- Partie entière et partie décimale sont séparées par un point (« . »)
- On peut utiliser l'écriture scientifique : 5e4 (soit 5 x 10^4 soit 50000)

Exemple



```
>>> type(-19.74)
  <class 'float'>
>>> .74
  0.74
>>> 19.
  19.0
>>> 1.974e2 # Ecriture scientifique
  197.4
```

Opérations sur les entiers/flottants

Opérations arithmétiques

- `-` : moins unaire
- `+`, `-`, `*`, `/` et `()` : opérations usuelles
- `//` : division entière
- `%` : reste de la division (modulo)
- `**` : puissance

Opérations sur les entiers/flottants

Exemple



```
>>> --5
  5
>>> (11-6)/2
  2.5
>>> 44//6
  7
>>> 40.5 // 8.5
  4.0
>>> 16 % 7
  2
>>> 8 % 2.5
  0.5
>>> 10**4
  10000
>>> 16**(1/2)
  4
```

Opérations sur les entiers/flottants

Opérations de comparaisons

- == : égalité
- <, >, != : inégalités strictes (inférieur, supérieur et différent)
- <=, >= : inégalités larges (inférieur ou égal, supérieur ou égal)

Opérations sur les entiers/flottants

Exemple



```
>>> 8*3-4 > 18
  True
>>> 18/2 < 9
  False
>>> 18/2 <= 9
  True
>>> 18/2 >= 9
  True
>>> 5.0 == 5
  True
>>> 5/2*3 != 5/(2*3) # Equivalent à 7.5 != 0.83333333
  True
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- **Booléens**
- Complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie



Booléens

Le type « bool »

- Permet de représenter les valeurs logiques VRAI et FAUX
- Deux mots réservés en Python : True et False
- N'importe qu'elle donnée en Python possède une valeur logique :
  - Les valeurs non nulles valent True
  - Les valeurs vides ou nulles valent False

Exemple



```
>>> type(True)
  <class 'bool'>
>>> type(False)
  <class 'bool'>
```

Opérations sur les booléens

Opérateurs booléens :

- not : négation
- and : et
- or : ou inclusif

Exemple



```
>>> not True
  False
>>> True and False
  False
>>> True or False
  True
>>> (7>6) and (3.0 == 3)
  True
```

Priorités des opérateurs

Opérateurs rangés par priorité descendante

- () : parenthèses (permettent de changer la priorité)
- - : moins unaire
- ** : puissance (opérateur associatif à droite)
- *, /, // et % : produit, quotient, division entière et modulo
- +, - : somme et différence
- <, >, <=, >=, == et !=
- not : négation
- and : « et » logique
- or : « ou » logique



Exemple

```
>>> not 16/2**2+10 != 15 % 8 * 2 and 0.5**-2/2*5 >= 10**2**(1/2)
  False
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- **Complexes**
- Chaînes de caractères
- Transtypage
- Variables et affectation
- Entrée/sortie



Complexes

Le type « complex »

- Permet de représenter n'importe quel nombre complexe
- Un nombre complexe s'écrit sous la forme z = a + jb avec :
  - a et b deux réels (a partie réelle de z et b partie imaginaire de z)
  - et $j^2 = -1$ (j est l'imaginaire pur)

Exemple



```
>>> type(1.5 + 2j)
  <class 'complex'>
>>> 1j * 1j # j² = -1
  -1 + 0j
>>> z = 2 + 3j
>>> z.real # Partie réelle de z
  2.0
>>> z.imag # Partie imaginaire de z
  3.0
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- **Chaînes de caractères**
- Transtypage
- Variables et affectation
- Entrée/sortie



Chaînes de caractères

Le type « str »

- Permet de représenter les textes
- Il s'agit de toute suite de caractères entourée par :
  - des apostrophes (')
  - des guillemets (")
  - ou bien des triples quotes (''' ou """)

Exemple



```
>>> type('Python')
  <class 'str'>
>>> 'Ceci est une chaîne de caractères'
  'Ceci est une chaîne de caractères'
>>> "Une autre chaîne"
  'Une autre chaîne'
```

Opération sur les chaînes de caractères

Caractères spéciaux

- \ : caractère d'échappement (transforme le caractèresuivant)
- \n : saut de ligne
- \t : tabulation
- \b : retour arrière

Exemple



```
>>> print('Bonjour\nà\ntous')
  Bonjour
  à
  tous
>>> print('Bonjour\tà\ttous')
  Bonjour    à    tous
>>> print('Bonjour\bà\btous')
  Bonjoutous
```

Opération sur les chaînes de caractères

Opérateurs

- + : concaténation
- * : répétition
- **len** : fonction retournant la longueur d'une chaîne

Exemple



```
>>> 'Introduction' + ' au ' + ' langage Python !'
  Introduction au langage Python !
>>> 'Python'*3
  'PythonPythonPython'
>>> len('Le langage ' + 'Python'*2)
   23
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- **Transtypage**
- Variables et affectation
- Entrée/sortie



Transtypage

Il est possible de tranformer le type d'une donnée :

- Tranformer en entier : fonction **int**()
- Tranformer en flottant : fonction **float**()
- Tranformer en chaîne de caractères : fonction **str**()

Cela fonctionne que si la donnée de départ est compatible avec le type d'arrivée

Transtypage

Exemple



```
>>> int(3.14) # Réel en entier
  3
>>> int('3') # Entier sous forme de chaîne de caractères en entier
  3
>>> int('3.14') # Réel en entier → erreur selon la version de Python
  ValueError: invalid literal for int() with base 10: '3.14'
>>> float(2) # Entier en réel
  2.0
>>> float('3.14') # Réel sous forme de chaîne de caractères en réel
  3.14
>>> float('3,14') # Erreur de syntaxe
  ValueError: could not convert string to float: '3,14'
>>> str(3.14) # Réel en chaîne de caractères
  '3.14'
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Entiers/flottants
- Booléens
- Complexes
- Chaînes de caractères
- Transtypage
- **Variables et affectation**
- Entrée/sortie



Variables et affectation

Variable

- Pour stocker une donnée en mémoire on utilise une variable
  - Le nom d'une variable est une suite de caractères ne contenant que des caractères alphanumériques (0 à 9, a à z, A à Z) ou le caractère souligné (_)
  - Le nom d'une variable commence forcément par une lettre ou un caractère souligné
  - Le nom d'une variable ne peut pas être un mot réservé du langage : **if**, **elif**, **else**, **while**, **not**, **and**, **or**, …
  - La casse est prise en compte (majuscule != minuscule)

- Stocker une donnée dans une variable se fait à l'aide de l'opérateur =

- Cette opération s'appelle une affectation

Variables et affectation

Exemple



```
>>> x = 1
>>> y = 2
>>> print(x,y)
  1 2
>>> chaine = 'Python'
>>> print(chaine*2)
  PythonPython
>>> _one_ = 1
>>> _2_ = 2
>>> print(_one_ + _2_)
  3
>>> 3eme = 3
  File "<stdin>", line 1
     3eme = 3
         ^
  SyntaxError: invalid syntax
```

Variables et affectation

L'opérateur =

- Il n'est pas symétrique (3 = a n'est pas valide)
- Il est associatif à droite (a = b = c signifie a = (b = c))
- Il peut servir pour des affectations parallèles (a,b = 1,2)

Exemple



```
>>> x,y,z = 1,2,3
>>> print(x,y,z)
  1 2 3
>>> x = y = z
>>> print(x,y,z)
  3 3 3
```

Où en sommes-nous ?

- Origines du langage Python
- Informations pratiques
- Données informatiques
- Commentaires
- Types de données standards
- Les entiers/flottants
- Les booléens
- Les complexes
- Chaînes de caractères
- Transtypage
- Variables et affectation
- **Entrée/sortie**



Entrée/sortie

Afficher des données : la fonction **print**

- Affiche la valeur des données passées en paramètres
- S'il y a plusieurs données, il faut les séparer par des virgules, elles seront séparées par un espace à l'affichage
- Interprète les caractères spéciaux (saut de ligne, tabulation, …)
- Se termine par un saut de ligne…
- … qui peut être modifié à l'aide du paramètre optionnel end



Exemple

```
>>> print('Hello');print('World !')
  Hello
  World !
>>> print('Hello', end=' ');print('World !')
  Hello World !
```

Entrée/sortie

Saisir des données au clavier : la fonction **input**

- Stoppe l'exécution du programme jusqu'à ce que l'utilisateur ait saisi une donnée
- La saisie se termine toujours par l'appui sur la touche Entrée
- Renvoie une donnée de type str
- Un texte peut être ajouté avant l'invite de saisie en paramètre de la fonction



Exemple

```
>>> age = input('Quel est votre âge ? ')
  Quel est votre âge ? 30
>>> age
  '30'
>>> print('Vous avez', age, 'ans')
  Vous avez 30 ans
```

